

Universidad del Valle

Escuela de Ingeniería de Sistemas y Computación

Programación por Restricciones

Taller 1: Modelamiento e Implementación de CSPs

Integrantes:

Cristian David Cabrera Pantoja
Junior Orlando Cantor Arevalo
Juan Diego Cárdenas Mejía

Profesor: Juan Francisco Díaz Frías

Cali, Colombia
4 de noviembre de 2026

Índice

1. Problema 1: Sudoku **1**
- 1.1. Descripción del Problema y Abstracción CSP . . . 1
  - 1.1.1. Parámetros y Datos de Entrada . . . 1
  - 1.1.2. Variables de Decisión y Dominios . . . 1
  - 1.1.3. Restricciones del Modelo . . . 1
- 1.2. Detalles de Implementación (`sudoku.mzn`) . . . 1
  - 1.2.1. Diferentes Implementaciones de Restricciones . . . 1
  - 1.2.2. Restricciones Redundantes y Rompimiento de Simetrías . . . 1
- 1.3. Estrategias de Búsqueda y Distribución . . . 1
- 1.4. Pruebas y Análisis Experimental . . . 1
- 1.5. Conclusiones del Ejercicio . . . 1

2. Problema 2: Kakuro **2**
- 2.1. Descripción del Problema y Abstracción CSP . . . 2
  - 2.1.1. Parámetros y Datos de Entrada . . . 2
  - 2.1.2. Variables de Decisión y Dominios . . . 3
  - 2.1.3. Restricciones del Modelo . . . 3
- 2.2. Detalles de Implementación (`kakuro.mzn`) . . . 4
  - 2.2.1. Variantes en el Modelado de Restricciones . . . 4
  - 2.2.2. Restricciones Redundantes y Rompimiento de Simetrías . . . 4
- 2.3. Estrategias de Búsqueda y Distribución . . . 5
- 2.4. Pruebas y Análisis Experimental . . . 5
- 2.5. Conclusiones del Ejercicio . . . 6

3. Problema 3: Secuencia Mágica **7**
- 3.1. Descripción del Problema y Abstracción CSP . . . 7
  - 3.1.1. Parámetros y Datos de Entrada . . . 7
  - 3.1.2. Variables de Decisión y Dominios . . . 7
  - 3.1.3. Restricciones Básicas . . . 7
- 3.2. Detalles de Implementación (`secuencia.mzn`) . . . 7
  - 3.2.1. Restricciones Redundantes y Demostración Teórica . . . 7
  - 3.2.2. Rompimiento de Simetrías . . . 7
- 3.3. Estrategias de Búsqueda y Distribución . . . 7
- 3.4. Pruebas y Análisis Experimental . . . 7
- 3.5. Conclusiones del Ejercicio . . . 7

4. Problema 4: Acertijo Lógico **8**
- 4.1. Descripción del Problema y Abstracción CSP . . . 8
  - 4.1.1. Pistas del Problema . . . 8
  - 4.1.2. Variables de Decisión y Dominios . . . 8
  - 4.1.3. Restricciones del Modelo . . . 8
- 4.2. Detalles de Implementación (`acertijo.mzn`) . . . 8
  - 4.2.1. Restricciones Redundantes y Rompimiento de Simetrías . . . 8
- 4.3. Estrategias de Búsqueda y Distribución . . . 8
- 4.4. Pruebas y Resultados . . . 8
- 4.5. Conclusiones del Ejercicio . . . 8

5. Problema 5: Ubicación de Personas en una Reunión **9**
5.1. Descripción del Problema y Abstracción CSP . . . 9
5.1.1. Parámetros y Estructura de Entrada . . . 9
5.1.2. Variables de Decisión y Dominios . . . 9
5.1.3. Restricciones del Modelo . . . 9
5.2. Detalles de Implementación (`reunion.mzn`) . . . 9
5.2.1. Restricciones Redundantes y Rompimiento de Simetrías . . . 9
5.3. Estrategias de Búsqueda y Distribución . . . 9
5.4. Pruebas y Análisis Experimental . . . 9
5.5. Conclusiones del Ejercicio . . . 9

6. Problema 6: Construcción de un Rectángulo **10**
6.1. Descripción del Problema y Abstracción CSP . . . 10
6.1.1. Parámetros y Datos de Entrada . . . 10
6.1.2. Variables de Decisión y Dominios . . . 10
6.1.3. Restricciones del Modelo . . . 11
6.2. Detalles de Implementación (`rectangulo.mzn`) . . . 11
6.2.1. Restricciones Redundantes y Rompimiento de Simetrías . . . 11
6.3. Estrategias de Búsqueda y Distribución . . . 13
6.4. Pruebas y Análisis Experimental . . . 13
6.5. Conclusiones del Ejercicio . . . 14

7. Conclusiones Generales **16**

1. Problema 1: Sudoku

1.1. Descripción del Problema y Abstracción CSP

1.1.1. Parámetros y Datos de Entrada

1.1.2. Variables de Decisión y Dominios

1.1.3. Restricciones del Modelo

1.2. Detalles de Implementación (`sudoku.mzn`)

1.2.1. Diferentes Implementaciones de Restricciones

1.2.2. Restricciones Redundantes y Rompimiento de Simetrías

1.3. Estrategias de Búsqueda y Distribución

1.4. Pruebas y Análisis Experimental

Instancia	Estrategia	Tiempo (s)	Nodos explorados	Nodos fallidos
Instancia 1	Default	–	–	–
Instancia 1	Custom 1	–	–	–
Instancia 2	Default	–	–	–

Tabla 1: Comparación de rendimiento para el problema del Sudoku.

1.5. Conclusiones del Ejercicio

2. Problema 2: Kakuro

2.1. Descripción del Problema y Abstracción CSP

El Kakuro (también conocido como *Cross Sums*) es un rompecabezas numérico de lógica combinatoria equivalente a un crucigrama matemático. El tablero está conformado por celdas negras (no jugables, que actúan como separadores y contienen las pistas numéricas) y celdas blancas (jugables). Una sucesión continua de casillas blancas adyacentes en dirección horizontal o vertical constituye un *tramo* o *serie*.

El objetivo consiste en asignar un dígito entero del 1 al 9 a cada casilla blanca del tablero de modo que se satisfagan simultáneamente dos condiciones fundamentales en cada serie horizontal y vertical:

1. La suma aritmética de los dígitos asignados a las casillas de la serie debe ser exactamente igual al valor de la pista asociada a dicho tramo.

2. Ningún dígito puede repetirse dentro de una misma serie (horizontal o vertical), aunque un mismo dígito sí puede aparecer varias veces en una misma fila o columna si sus apariciones están separadas por al menos una casilla negra.

Desde la perspectiva de la Programación por Restricciones (CSP), el Kakuro se abstrae como un problema de satisfacción sobre dominios finitos enteros donde cada celda blanca corresponde a una variable de decisión restringida por la intersección de una restricción lineal de suma y una restricción de disyunción por pares (o restricción global de diferencia) en su tramo horizontal y en su tramo vertical.

2.1.1. Parámetros y Datos de Entrada

Siguiendo una representación matricial paramétrica que separa completamente el modelo (`kakuro.mzn`) de los datos de cada instancia (`.dzn`), se definen los siguientes parámetros de entrada:

- $F \in \mathbb{Z}^+$: Número de filas del área jugable del tablero.
- $C \in \mathbb{Z}^+$: Número de columnas del área jugable del tablero.
- $n_H \in \mathbb{Z}^+$: Número total de series (tramos) horizontales.
- $n_V \in \mathbb{Z}^+$: Número total de series (tramos) verticales.
- $P^H \in \mathbb{Z}^{n_H \times 4}$: Matriz de pistas horizontales (`pistas_h`), donde cada fila $k \in \{1, \dots, n_H\}$ representa la tupla $(f_k, c_k^{\text{ini}}, c_k^{\text{fin}}, S_k^H)$, indicando que en la fila f_k el tramo horizontal va desde la columna c_k^{ini} hasta la columna c_k^{fin} y debe sumar S_k^H.
- $P^V \in \mathbb{Z}^{n_V \times 4}$: Matriz de pistas verticales (`pistas_v`), donde cada fila $m \in \{1, \dots, n_V\}$ representa la tupla $(c_m, f_m^{\text{ini}}, f_m^{\text{fin}}, S_m^V)$, indicando que en la columna c_m el tramo vertical va desde la fila f_m^{ini} hasta la fila f_m^{fin} y debe sumar S_m^V.

A partir de las pistas horizontales se define estáticamente el conjunto de posiciones de casillas blancas $B \subseteq \{1, \dots, F\} \times \{1, \dots, C\}$ como:

$$B = \left\{ (i,j) \in \{1,\dots,F\} \times \{1,\dots,C\} \;\middle|\; \exists k \in \{1,\dots,n_H\} : f_k = i \wedge c_k^{\text{ini}} \leq j \leq c_k^{\text{fin}} \right\}$$

2.1.2. Variables de Decisión y Dominios

Se define una matriz bidimensional de variables de decisión $X_{i,j}$ (`tablero[i,j]`) para cada celda $(i, j) \in \{1, \ldots, F\} \times \{1, \ldots, C\}$. Con el fin de establecer el dominio más pequeño posible para cada tipo de casilla sin recurrir a tipos opcionales:

- Para toda casilla blanca $(i, j) \in B$, la variable representa el dígito escrito en la celda y su dominio efectivo es:

$$D(X_{i,j}) = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}, \quad \forall (i, j) \in B$$

- Para toda casilla negra $(i, j) \notin B$, la variable se fija mediante una restricción unaria al valor constante cero, reduciendo su dominio a un único elemento:

$$D(X_{i,j}) = \{0\}, \quad \forall (i, j) \notin B$$

2.1.3. Restricciones del Modelo

El modelo matemático principal se compone de las siguientes familias de restricciones:

1. **Partición de dominios entre casillas blancas y negras:**

$$\forall i \in \{1, \ldots, F\},\ \forall j \in \{1, \ldots, C\} : \quad \begin{cases} X_{i,j} \geq 1 & \text{si } (i, j) \in B \\ X_{i,j} = 0 & \text{si } (i, j) \notin B \end{cases}$$

2. **Suma exacta en cada serie horizontal:**

$$\forall k \in \{1, \ldots, n_H\} : \quad \sum_{j=c_k^{\text{ini}}}^{c_k^{\text{fin}}} X_{f_k,j} = S_k^H$$

3. **Suma exacta en cada serie vertical:**

$$\forall m \in \{1, \ldots, n_V\} : \quad \sum_{i=f_m^{\text{ini}}}^{f_m^{\text{fin}}} X_{i,c_m} = S_m^V$$

4. **No repetición de dígitos por tramo (Formulación A — Desigualdades binarias):**

$$\forall k \in \{1, \ldots, n_H\},\ \forall j_1, j_2 \in \{c_k^{\text{ini}}, \ldots, c_k^{\text{fin}}\} \text{ con } j_1 < j_2 : \quad X_{f_k,j_1} \neq X_{f_k,j_2}$$

$$\forall m \in \{1, \ldots, n_V\},\ \forall i_1, i_2 \in \{f_m^{\text{ini}}, \ldots, f_m^{\text{fin}}\} \text{ con } i_1 < i_2 : \quad X_{i_1,c_m} \neq X_{i_2,c_m}$$

5. **No repetición de dígitos por tramo (Formulación B — Restricción global `alldifferent`):**

$$\forall k \in \{1, \ldots, n_H\} : \quad \text{alldifferent}\left(\left[X_{f_k,j} \;\middle|\; j \in c_k^{\text{ini}} \ldots c_k^{\text{fin}}\right]\right)$$

$$\forall m \in \{1, \ldots, n_V\} : \quad \text{alldifferent}\left(\left[X_{i,c_m} \;\middle|\; i \in f_m^{\text{ini}} \ldots f_m^{\text{fin}}\right]\right)$$

2.2. Detalles de Implementación (`kakuro.mzn`)

La implementación en `kakuro.mzn` se estructuró de manera completamente paramétrica:

- **Validación temprana de datos (`assert`):** Antes de instanciar las restricciones, se verifica que las dimensiones F, C, n_H, n_V sean positivas y que la suma total de las pistas horizontales coincida exactamente con la suma total de las pistas verticales ($\sum_k S_k^H = \sum_m S_m^V$), previniendo errores de transcripción en los archivos `.dzn`.
- **Función booleana de pertenencia (`es_blanca`):** Se definió una función auxiliar determinista (`par bool`) que evalúa mediante un cuantificador existencial (`exists`) si la coordenada (i, j) pertenece a algún tramo horizontal. Esto permite tanto acotar los dominios del tablero como construir por comprensión el arreglo unidimensional `vars_blancas` que contiene exclusivamente las variables de decisión de las casillas blancas, evitando que las estrategias de búsqueda ramifiquen trivialmente sobre las casillas negras fijadas en 0.
- **Banderas de control experimental:** Se incorporaron los parámetros booleanos `usar_alldifferent` y `usar_redundantes` para activar o desactivar las variantes de modelado y evaluar su impacto real sobre el número de propagadores y nodos.

2.2.1. Variantes en el Modelado de Restricciones

Se implementaron y contrastaron dos enfoques para garantizar la unicidad de dígitos en cada tramo de longitud L:

1. **Descomposición en desigualdades binarias ($\neq$):** Genera $\binom{L}{2} = \frac{L(L-1)}{2}$ restricciones binarias independientes por cada serie. Su desventaja principal en propagación es su carácter estrictamente local: una desigualdad $X_a \neq X_b$ permanece inactiva hasta que una de las dos variables queda instanciada a un único valor (consistencia de arco débil sobre $\neq$), por lo que no detecta conflictos de cardinalidad cuando varias variables comparten un subconjunto reducido de valores.
2. **Restricción global `alldifferent`:** Agrupa todas las variables de la serie en un único propagador global por tramo, reduciendo sustancialmente el número de propagadores en el grafo de restricciones y permitiendo detectar subconjuntos saturados de Hall antes de que las variables estén fijas.

2.2.2. Restricciones Redundantes y Rompimiento de Simetrías

Restricciones Redundantes. En el modelo básico, el propagador de la suma lineal $\sum_{r=1}^{L} X_r = S$ desconoce que las variables X_r deben tomar valores distintos entre sí; por ello, calcula sus cotas asumiendo que las $L-1$ variables restantes podrían repetir el valor mínimo 1 o el valor máximo 9. Para acoplar semánticamente la restricción de suma con la restricción `alldifferent`, se dedujeron e implementaron dos familias de restricciones redundantes:

1. **Cotas combinatorias extremas por longitud de serie:** En un tramo de longitud $L = \text{fin} - \text{ini} + 1$ y suma objetivo S, las otras $L-1$ casillas son dígitos distintos en $\{1, \ldots, 9\}$. Por tanto, la suma mínima posible de las otras $L-1$ casillas es $\sum_{r=1}^{L-1} r = \frac{(L-1)L}{2}$, y su suma máxima posible es $\sum_{r=11-L}^{9} r = \frac{(L-1)(20-L)}{2}$. En consecuencia, toda variable X perteneciente a dicho tramo debe satisfacer obligatoriamente:

$$S - \frac{(L-1)(20-L)}{2} \leq X \leq S - \frac{(L-1)L}{2}$$

Por ejemplo, en una serie de longitud $L = 2$ con suma $S = 4$, esta cota deduce de inmediato en el nodo raíz que $X \leq 4 - 1 = 3$, eliminando los valores $\{4, 5, 6, 7, 8, 9\}$ antes de iniciar

la búsqueda. En el preprocesamiento de MiniZinc, esta poda de dominios permite incluso fijar variables de manera directa y simplificar propagadores enteros.

2. **Invariante de suma global del tablero:** Dado que las series horizontales particionan el conjunto de casillas blancas B, la suma de todas las celdas del tablero debe coincidir con la suma acumulada de todas las pistas horizontales:

$$\sum_{i=1}^{F}\sum_{j=1}^{C} X_{i,j} = \sum_{k=1}^{n_H} S_k^H$$

Análisis de Simetrías. A diferencia de problemas de empaquetamiento o asignación con objetos indistinguibles, el problema de Kakuro **no presenta simetrías de variables ni de valores**. Cada celda blanca $(i, j) \in B$ está identificada de forma única por la intersección de una serie horizontal específica (con suma S_k^H) y una serie vertical específica (con suma S_m^V). Si se permutaran los valores de dos casillas dentro de un mismo tramo horizontal, se alterarían las sumas de los tramos verticales ortogonales que se cruzan en dichas columnas. Asimismo, en un tablero de Kakuro bien propuesto la solución es única, por lo que no existen soluciones simétricas equivalentes ni se requieren restricciones de rompimiento de simetrías.

2.3. Estrategias de Búsqueda y Distribución

Sobre el arreglo unidimensional `vars_blancas` (que filtra únicamente las variables con $(i, j) \in B$) se evaluaron seis configuraciones de búsqueda mediante `int_search`:

1. **Default (`solve satisfy`):** Estrategia nativa por defecto del solver Gecode.
2. **`input_order` + `indomain_min`:** Selecciona las casillas blancas en orden lexicográfico de lectura (fila por fila, de izquierda a derecha) y prueba primero el menor dígito disponible.
3. **`input_order` + `indomain_max`:** Selecciona las casillas en orden de fila pero asigna primero el mayor dígito disponible en el dominio.
4. **`first_fail` + `indomain_min`:** Selecciona dinámicamente la casilla blanca con el dominio actual más pequeño (principio de fallo primero), priorizando las intersecciones de tramos cortos o sumas extremas, y prueba el menor valor disponible.
5. **`first_fail` + `indomain_split`:** Combina la selección de variable más restringida con la bisección de su dominio ($\leq m$ vs. $> m$).
6. **`dom_w_deg` + `indomain_min`:** Selecciona la variable según la razón entre el tamaño de su dominio y su grado ponderado de fallos acumulados en los propagadores asociados.

2.4. Pruebas y Análisis Experimental

Se construyó una batería de tres instancias en archivos de datos independientes:

- `kakuro1.dzn` **(Tablero 1 del taller):** Área jugable de 5×4 con 14 casillas blancas, 5 series horizontales y 4 series verticales (suma total = 65).
- `kakuro2.dzn` **(Tablero 2 del taller):** Área jugable de 7×7 con 24 casillas blancas dispuestas en anillo diagonal, 9 series horizontales y 9 series verticales (suma total = 140).
- `kakuro3.dzn` **(Tablero 3 adicional de mayor escala):** Área jugable de 8×8 con 36 casillas blancas distribuidas en bandas diagonales, 16 series horizontales y 16 series verticales (suma total = 205).

Las tres instancias poseen solución única verificada mediante búsqueda exhaustiva (`--all-solutions`). Para los dos tableros del enunciado del taller, las matrices solución obtenidas por el modelo son:

$$\text{Kakuro 1 } (5\times 4) = \begin{pmatrix} 2 & 6 & 1 & \cdot \\ 1 & 9 & 3 & \cdot \\ 6 & 7 & \cdot & \cdot \\ \cdot & 4 & 2 & 1 \\ \cdot & 8 & 9 & 2 \end{pmatrix}, \qquad \text{Kakuro 2 } (7\times 7) = \begin{pmatrix} \cdot & \cdot & 2 & 7 & \cdot & \cdot & \cdot \\ \cdot & \cdot & 7 & 8 & 9 & \cdot & \cdot \\ 2 & 4 & 1 & \cdot & 4 & 8 & \cdot \\ 5 & 7 & \cdot & \cdot & \cdot & 9 & 1 \\ \cdot & 9 & 5 & \cdot & 1 & 6 & 2 \\ \cdot & \cdot & 9 & 8 & 5 & \cdot & \cdot \\ \cdot & \cdot & \cdot & 9 & 2 & \cdot & \cdot \end{pmatrix}$$

En el Cuadro 2 se presenta el estudio de ablación de las tres variantes de modelado al explorar la totalidad del espacio de búsqueda (`--all-solutions` con estrategia `first_fail + indomain_min`) para encontrar la solución y demostrar su unicidad. Se observa con claridad cómo el paso de desigualdades binarias ($\neq$) a `alldifferent` reduce el número de propagadores entre un 28 % y un 49 %, mientras que la incorporación de las cotas combinatorias redundantes poda los dominios desde la etapa de aplanamiento, reduciendo los propagadores activos hasta en un 87,5 % (de 104 a 13 en `kakuro3.dzn`) y disminuyendo las propagaciones y nodos en más de la mitad.

Instancia	Variante de Modelado	Propagadores	Propagaciones	Nodos	Fallos	Tiempo (ms)
`kakuro1.dzn`	A: Desigualdades ($\neq$) sin red.	39	151	9	4	0.13
(5×4, 14 blancas)	B: `alldifferent` sin red.	20	134	9	4	0.14
	C: `alldifferent` + Redundantes	**13**	**57**	**5**	**2**	**0.13**
`kakuro2.dzn`	A: Desigualdades ($\neq$) sin red.	56	164	9	4	0.16
(7×7, 24 blancas)	B: `alldifferent` sin red.	40	158	9	4	0.15
	C: `alldifferent` + Redundantes	**9**	**62**	**3**	**1**	**0.12**
`kakuro3.dzn`	A: Desigualdades ($\neq$) sin red.	104	336	11	5	0.22
(8×8, 36 blancas)	B: `alldifferent` sin red.	64	305	11	5	0.22
	C: `alldifferent` + Redundantes	**13**	**114**	**3**	**1**	**0.15**

Tabla 2: Comparación experimental de las variantes de modelado en Kakuro (búsqueda exhaustiva `--all-solutions`).

Por su parte, el Cuadro 3 compara el desempeño de las seis estrategias de búsqueda evaluadas para hallar la primera solución tanto sobre el modelo sin restricciones redundantes (Variante B) como sobre el modelo completo con restricciones redundantes (Variante C).

2.5. Conclusiones del Ejercicio

- **Sinergia entre restricciones globales y cotas redundantes:** Aunque reemplazar las desigualdades binarias ($\neq$) por la restricción global `alldifferent` reduce drásticamente el número de propagadores del modelo, el mayor salto de eficiencia se obtiene al añadir las cotas combinatorias redundantes $S - \frac{(L-1)(20-L)}{2} \leq X \leq S - \frac{(L-1)L}{2}$. Estas desigualdades inyectan el conocimiento de no repetición directamente sobre los límites de cada variable antes de iniciar la búsqueda, resolviendo `kakuro2.dzn` y `kakuro3.dzn` prácticamente en el nodo raíz (2 nodos y 0 fallos).
- **Comportamiento de las heurísticas de búsqueda:** Sin restricciones redundantes, en `kakuro1.dzn` las heurísticas dinámicas (`first_fail`) superan al orden estático (`input_order`) y a la búsqueda por defecto al elegir primero las casillas de tramos cortos (longitud 2), reduciendo la profundidad máxima del árbol de 5 a 3. Con las restricciones redundantes activas, los dominios iniciales quedan tan acotados que tanto `first_fail + indomain_min` como `first_fail + indomain_split` convergen de forma inmediata con una profundidad máxima de apenas 1 nivel.

		Sin Redundantes (Variante B)			Con Redundantes (Variante C)		
Instancia	Estrategia de Búsqueda	Tiempo (ms)	Nodos	Fallos	Tiempo (ms)	Nodos	Fallos
`kakuro1.dzn`	Default (`solve satisfy`)	0.12	10	4	0.17	4	1
	`input_order + indomain_min`	0.12	8	3	0.11	6	2
	`input_order + indomain_max`	0.14	7	2	0.13	4	1
	`first_fail + indomain_min`	0.12	7	2	0.15	5	2
	`first_fail + indomain_split`	0.12	7	2	**0.10**	5	2
	`dom_w_deg + indomain_min`	0.12	8	3	0.11	6	2
`kakuro2.dzn`	Default (`solve satisfy`)	0.13	4	0	0.14	2	0
	`input_order + indomain_min`	0.13	5	1	0.18	2	0
	`input_order + indomain_max`	0.14	8	3	0.12	3	1
	`first_fail + indomain_min`	0.15	8	3	**0.11**	**2**	**0**
	`first_fail + indomain_split`	0.21	8	3	0.16	2	0
	`dom_w_deg + indomain_min`	0.22	8	3	0.15	2	0
`kakuro3.dzn`	Default (`solve satisfy`)	0.18	7	1	0.21	3	0
	`input_order + indomain_min`	0.19	7	1	**0.16**	**2**	**0**
	`input_order + indomain_max`	0.22	10	4	0.24	3	1
	`first_fail + indomain_min`	0.36	7	1	0.24	**2**	**0**
	`first_fail + indomain_split`	0.19	7	1	0.19	**2**	**0**
	`dom_w_deg + indomain_min`	0.29	7	1	0.16	**2**	**0**

Tabla 3: Comparación experimental de estrategias de búsqueda en el modelo de Kakuro.

3. Problema 3: Secuencia Mágica

3.1. Descripción del Problema y Abstracción CSP

3.1.1. Parámetros y Datos de Entrada

3.1.2. Variables de Decisión y Dominios

3.1.3. Restricciones Básicas

3.2. Detalles de Implementación (`secuencia.mzn`)

3.2.1. Restricciones Redundantes y Demostración Teórica

3.2.2. Rompimiento de Simetrías

3.3. Estrategias de Búsqueda y Distribución

3.4. Pruebas y Análisis Experimental

n	Modelo	Tiempo (s)	Nodos explorados	Nodos fallidos	Soluciones
10	Sin redundantes	–	–	–	–
10	Con redundantes	–	–	–	–
20	Sin redundantes	–	–	–	–
20	Con redundantes	–	–	–	–

Tabla 4: Impacto de las restricciones redundantes en la Secuencia Mágica.

3.5. Conclusiones del Ejercicio

4. Problema 4: Acertijo Lógico

4.1. Descripción del Problema y Abstracción CSP

4.1.1. Pistas del Problema

1. Juan es más viejo que González, a quien le gusta la música clásica.
2. El fan de la música pop, que no es García, no tiene 24.
3. Oscar, quien no es López, tiene 25.
4. La música favorita de Darío no es el jazz.
5. Las edades de los 3 amigos están entre 24 y 26 años.

4.1.2. Variables de Decisión y Dominios

4.1.3. Restricciones del Modelo

4.2. Detalles de Implementación (`acertijo.mzn`)

4.2.1. Restricciones Redundantes y Rompimiento de Simetrías

4.3. Estrategias de Búsqueda y Distribución

4.4. Pruebas y Resultados

Nombre	Apellido	Edad	Género Musical Favorito
Juan	–	–	–
Oscar	–	25	–
Darío	–	–	–

Tabla 5: Solución identificada para el Acertijo Lógico.

4.5. Conclusiones del Ejercicio

5. Problema 5: Ubicación de Personas en una Reunión

5.1. Descripción del Problema y Abstracción CSP

5.1.1. Parámetros y Estructura de Entrada

5.1.2. Variables de Decisión y Dominios

5.1.3. Restricciones del Modelo

5.2. Detalles de Implementación (`reunion.mzn`)

5.2.1. Restricciones Redundantes y Rompimiento de Simetrías

5.3. Estrategias de Búsqueda y Distribución

5.4. Pruebas y Análisis Experimental

Instancia (N)	Estrategia	Tiempo (s)	Nodos explorados	Nodos fallidos
Instancia base ($N = 8$)	Default	–	–	–
Instancia base ($N = 8$)	Custom	–	–	–
Instancia grande ($N = 20$)	Default	–	–	–

Tabla 6: Resultados experimentales para el problema de la reunión.

5.5. Conclusiones del Ejercicio

6. Problema 6: Construcción de un Rectángulo

6.1. Descripción del Problema y Abstracción CSP

El problema de la construcción de un rectángulo consiste en ensamblar un conjunto dado de n cuadrados de lados enteros conocidos $s_1, s_2, \ldots, s_n$ para formar exactamente un rectángulo contenedor de anchura W y altura H. Para que una disposición sea válida, deben cumplirse tres condiciones geométricas:

1. **Integridad de uso:** Todos los n cuadrados deben ubicarse dentro del rectángulo contenedor con sus lados paralelos a los ejes coordenados.
2. **Contención:** Ningún cuadrado puede sobresalir de las fronteras $[0, W]\times[0, H]$ del rectángulo.
3. **No solapamiento:** Los interiores de dos cuadrados cualesquiera deben ser disjuntos (solo pueden compartir bordes o vértices).

Bajo el paradigma de Programación por Restricciones (CSP), si se sitúa el origen del sistema de coordenadas $(0, 0)$ en la esquina inferior izquierda del rectángulo contenedor, la posición de cada cuadrado $i \in \{1, \ldots, n\}$ queda completamente determinada por las coordenadas enteras (x_i, y_i) de su esquina inferior izquierda, ocupando la región espacial $[x_i, x_i + s_i] \times [y_i, y_i + s_i]$.

6.1.1. Parámetros y Datos de Entrada

El modelo paramétrico (`rectangulo.mzn`) recibe como entrada los siguientes datos desde los archivos `.dzn`:

- $n \in \mathbb{Z}^+$: Número total de cuadrados a ensamblar.
- $S = \{1, \ldots, n\}$: Conjunto de índices de los cuadrados.
- $s_i \in \mathbb{Z}^+$ (`lado[i]`): Longitud del lado del cuadrado $i \in S$. Por convención de modelado (verificada mediante `assert`), los cuadrados se suministran ordenados de mayor a menor tamaño: $s_1 \geq s_2 \geq \cdots \geq s_n$.
- $W \in \mathbb{Z}^+$: Anchura total del rectángulo contenedor.
- $H \in \mathbb{Z}^+$: Altura total del rectángulo contenedor.

6.1.2. Variables de Decisión y Dominios

Para cada cuadrado $i \in S = \{1, \ldots, n\}$ se definen dos variables de decisión enteras:

- x_i (`x[i]`): Coordenada horizontal (eje X) de la esquina inferior izquierda del cuadrado i.
- y_i (`y[i]`): Coordenada vertical (eje Y) de la esquina inferior izquierda del cuadrado i.

Dado que el cuadrado i se extiende s_i unidades hacia la derecha y s_i unidades hacia arriba, el dominio efectivo más estrecho para cada variable de decisión es:

$$D(x_i) = \{0, 1, \ldots, W - s_i\}, \qquad D(y_i) = \{0, 1, \ldots, H - s_i\}, \qquad \forall i \in \{1, \ldots, n\}$$

En MiniZinc, dado que la declaración de un arreglo requiere un rango de índice uniforme, los arreglos `x` e `y` se declaran sobre $0 \ldots W - \min_k(s_k)$ y $0 \ldots H - \min_k(s_k)$ respectivamente, e inmediatamente se acota el dominio exacto de cada componente mediante restricciones unarias.

6.1.3. Restricciones del Modelo

El modelo matemático base comprende las siguientes restricciones fundamentales:

1. **Restricciones unarias de contención exacta dentro del rectángulo $W \times H$:**

$$\forall i \in \{1, \ldots, n\}: \quad 0 \leq x_i \leq W - s_i \ \wedge \ 0 \leq y_i \leq H - s_i$$

2. **Restricción disyuntiva de no solapamiento bidimensional por pares:** Dos cuadrados distintos $i, j \in \{1, \ldots, n\}$ $(i < j)$ no se solapan si y solo si al menos una de las cuatro separaciones ortogonales (izquierda, derecha, abajo o arriba) se cumple:

$$\forall i, j \in \{1, \ldots, n\} \text{ con } i < j: \quad (x_i + s_i \leq x_j) \vee (x_j + s_j \leq x_i) \vee (y_i + s_i \leq y_j) \vee (y_j + s_j \leq y_i)$$

6.2. Detalles de Implementación (`rectangulo.mzn`)

Siguiendo el diseño modular de los problemas de planificación disyuntiva y mudanza vistos en el curso:

- **Predicado modular `no_solapar`:** La disyunción de cuatro cláusulas se encapsuló en un predicado reutilizable `no_solapar(x1, l1, y1, x2, l2, y2)`, invocándolo sobre el triángulo superior de pares $i < j$ para evitar duplicar restricciones entre el mismo par de cuadrados.
- **Ordenamiento decreciente de cuadrados y validación de área:** Mediante sentencias `assert` se verifica en tiempo de compilación que el arreglo `lado` cumpla $s_i \geq s_{i+1}$ y que la suma de áreas $\sum_{i=1}^{n} s_i^2$ coincida exactamente con el área $W \cdot H$ del contenedor.
- **Arreglo intercalado para búsqueda (`vars_xy`):** Se construyó por comprensión el arreglo unidimensional $[x_1, y_1, x_2, y_2, \ldots, x_n, y_n]$ de longitud $2n$, de modo que las heurísticas posicionales fijen de manera consecutiva ambas coordenadas (x_i, y_i) del cuadrado i antes de pasar al cuadrado $i + 1$.

6.2.1. Restricciones Redundantes y Rompimiento de Simetrías

Restricciones Redundantes. La restricción disyuntiva de no solapamiento $\vee$ sufre de propagación tardía (reificación débil): mientras un par de cuadrados conserve al menos dos direcciones posibles de separación, ninguna de las desigualdades individuales poda los dominios de x_i o y_i. Esto permite que el solver intente apilar varios cuadrados en una misma franja vertical u horizontal excediendo la altura H o la anchura W, descubriendo el fallo únicamente tras múltiples ramificaciones.

Para dotar al modelo de una visión global del espacio disponible, se observa que el empaquetamiento ortogonal 2D se proyecta sobre cada eje como un problema de planificación acumulativa de recursos (`cumulative`):

1. **Proyección acumulativa sobre el eje X:** Interpretando x_i como el instante de inicio de una tarea de duración s_i que consume una cantidad s_i del recurso vertical disponible, en toda abscisa $c \in [0, W)$ la suma de las alturas de los cuadrados que intersectan la recta vertical $X = c$ no puede exceder H (de hecho, como el recubrimiento es exacto, suma exactamente H):

$$\forall c \in \{0, \ldots, W-1\}: \quad \sum_{\substack{i=1 \\ x_i \leq c < x_i + s_i}}^{n} s_i \leq H \quad \Longleftrightarrow \quad \text{cumulative}(x, s, s, H)$$

2. **Proyección acumulativa sobre el eje Y:** De forma dual, interpretando y_i como el inicio de una tarea de duración s_i y consumo horizontal s_i, en toda ordenada $r \in [0, H)$ la suma de anchuras de los cuadrados que intersectan la recta horizontal $Y = r$ no puede exceder W:

$$\forall r \in \{0, \ldots, H-1\} : \quad \sum_{\substack{i=1 \\ y_i \le r < y_i + s_i}}^{n} s_i \le W \quad \iff \quad \text{cumulative}(y, s, s, W)$$

Rompimiento de Simetrías y Recuperación de Soluciones. El problema posee dos fuentes independientes de simetría que multiplican exponencialmente el número de soluciones equivalentes:

1. **Simetría de permutación entre cuadrados de igual tamaño:**

   - *Simetría que rompe:* Si un subconjunto de k_g cuadrados tiene exactamente el mismo lado ($s_i = s_{i+1} = \ldots$), sus ubicaciones en cualquier solución válida pueden permutarse de $k_g!$ maneras distintas produciendo el mismo diseño geométrico. Por ejemplo, en la instancia del taller lado $= [3, 2, 2, 1, 1, 1]$, existen $2! \times 3! = 12$ permutaciones idénticas por cada configuración geométrica.
   - *Restricción impuesta:* Para todo par consecutivo de cuadrados de igual lado ($s_i = s_{i+1}$), se impone un orden lexicográfico estricto sobre sus vectores de posición $(x_i, y_i) <_{\text{lex}} (x_{i+1}, y_{i+1})$:

     $$\forall i \in \{1, \ldots, n-1\} \text{ tal que } s_i = s_{i+1} : \quad (x_i < x_{i+1}) \ \vee \ (x_i = x_{i+1} \wedge y_i < y_{i+1})$$

   - *Recuperación de las soluciones descartadas:* A partir de una solución canónica hallada, todas las soluciones equivalentes por reetiquetado se recuperan aplicando las $\prod_g k_g!$ permutaciones de índices dentro de cada clase de cuadrados de igual tamaño.

2. **Simetría geométrica dihedral del rectángulo contenedor:**

   - *Simetría que rompe:* Cualquier recubrimiento válido del rectángulo $W \times H$ permanece válido al aplicarle una reflexión horizontal respecto a $X = W/2$, una reflexión vertical respecto a $Y = H/2$, o una rotación de 180° (composición de ambas), generando 4 soluciones congruentes (y hasta 8 si el contenedor es cuadrado, $W = H$, al incluir la reflexión diagonal $X \leftrightarrow Y$).
   - *Restricción impuesta:* Se restringe el centro del cuadrado de mayor tamaño ($i = 1$) para que se ubique en el cuadrante inferior izquierdo del rectángulo contenedor:

     $$2x_1 \le W - s_1 \quad \wedge \quad 2y_1 \le H - s_1$$

     y adicionalmente, cuando el contenedor es cuadrado ($W = H$), se fuerza $x_1 \le y_1$ para romper la simetría de transposición diagonal.
   - *Recuperación de las soluciones descartadas:* Dada una solución canónica $\{(x_i, y_i)\}_{i=1}^{n}$, las configuraciones geométricas simétricas se reconstruyen mediante las isometrías afines:

     $$\begin{aligned} \text{Reflexión horizontal:} \quad & (x_i', y_i') = (W - s_i - x_i, \ y_i) \\ \text{Reflexión vertical:} \quad & (x_i', y_i') = (x_i, \ H - s_i - y_i) \\ \text{Rotación de } 180° : \quad & (x_i', y_i') = (W - s_i - x_i, \ H - s_i - y_i) \end{aligned}$$

     (y si $W = H$, componiendo las cuatro transformaciones anteriores con el intercambio de ejes (y_i, x_i)).

6.3. Estrategias de Búsqueda y Distribución

Para evaluar el impacto del orden de ramificación sobre el empaquetamiento 2D, se compararon seis estrategias de búsqueda:

1. **Default (`solve satisfy`):** Heurística nativa del solver Gecode.
2. **`input_order` + `indomain_min` sobre `vars_xy`:** Dado que `lado` está ordenado de mayor a menor ($s_1 \geq \cdots \geq s_n$) y `vars_xy` intercala $[x_1, y_1, \ldots, x_n, y_n]$, esta estrategia implementa la heurística geométrica clásica *Largest-Square-First, Bottom-Left*: posiciona primero los cuadrados más voluminosos lo más abajo y a la izquierda posible.
3. **`input_order` + `indomain_max` sobre `vars_xy`:** Ubica los cuadrados de mayor a menor intentando colocarlos en la esquina superior derecha disponible.
4. **`first_fail` + `indomain_min` sobre `vars_xy`:** Selecciona dinámicamente la coordenada x_i o y_i con menor dominio actual.
5. **`seq_search` por ejes (X luego Y) con `first_fail`:** Fija primero todas las abscisas $x_1, \ldots, x_n$ de todos los cuadrados y posteriormente resuelve sus ordenadas $y_1, \ldots, y_n$.
6. **`dom_w_deg` + `indomain_split` sobre `vars_xy`:** Combina el grado ponderado de fallos con bisección de dominios.

6.4. Pruebas y Análisis Experimental

Se diseñaron tres instancias de prueba en archivos `.dzn`:

- `rectangulo1.dzn` **(Instancia del enunciado del taller):** $W = 5, H = 4$ (Área = 20), $n = 6$ cuadrados con lado = $[3, 2, 2, 1, 1, 1]$. Tiene $2! \times 3! \times 4 = 48$ soluciones simétricas equivalentes a una única configuración geométrica salvo isometrías.
- `rectangulo2.dzn` **(Instancia mediana):** $W = 8, H = 5$ (Área = 40), $n = 9$ cuadrados con lado = $[3, 3, 2, 2, 2, 2, 2, 1, 1]$. Cada configuración geométrica genera $2! \times 5! \times 2! \times 4 =$ 1,920 soluciones simétricas.
- `rectangulo3.dzn` **(Instancia grande):** $W = 10, H = 7$ (Área = 70), $n = 12$ cuadrados con lado = $[4, 4, 3, 3, 2, 2, 2, 2, 1, 1, 1, 1]$.

Para la instancia del taller (`rectangulo1.dzn`, 5×4), el modelo obtiene la solución canónica única:

Cuadrado i	1 (3×3)	2 (2×2)	3 (2×2)	4 (1×1)	5 (1×1)	6 (1×1)
Posición (x_i, y_i)	$(0, 0)$	$(3, 0)$	$(3, 2)$	$(0, 3)$	$(1, 3)$	$(2, 3)$

$$\text{Mapa } (5 \times 4) = \begin{pmatrix} 4 & 5 & 6 & 3 & 3 \\ 1 & 1 & 1 & 3 & 3 \\ 1 & 1 & 1 & 2 & 2 \\ 1 & 1 & 1 & 2 & 2 \end{pmatrix}$$

En el Cuadro 7 se presenta el estudio de ablación al buscar todas las soluciones (`--all-solutions`, límite de 5,000 ms) bajo las cuatro combinaciones del modelo: (1) Base (solo `no_solapar`), (2) Base + Redundantes (`cumulative`), (3) Base + Rompimiento de Simetrías y (4) Modelo Completo. Los resultados demuestran el efecto multiplicativo de ambas técnicas: en `rectangulo1.dzn`, el rompimiento de simetrías reduce exactamente las 48 soluciones simétricas a la única solución canónica (1 solución), disminuyendo los nodos de 335 a 15. En `rectangulo2.dzn` y `rectangulo3.dzn`, donde los modelos sin rompimiento de simetrías agotan el tiempo límite de 5 s debido a la explosión factorial de permutaciones, el rompimiento de simetrías permite

enumerar la totalidad de las soluciones canónicas (8 y 122 respectivamente) en milisegundos, y la adición de `cumulative` reduce a menos de la mitad los nodos y fallos restantes (1,505 → 753 nodos en `rectangulo2.dzn` y 21,755 → 9,649 nodos en `rectangulo3.dzn`).

Instancia	Configuración del Modelo	Soluciones	Propagaciones	Nodos	Fallos	Tiempo (ms)
`rectangulo1.dzn`	1. Base (solo `no_solapar`)	48	6,146	335	120	1.37
$(5 \times 4, n = 6)$	2. Base + Redundantes (`cumulative`)	48	5,920	255	80	1.48
	3. Base + Simetrías	1	413	17	8	0.14
	4. Completo (Redundantes + Simetrías)	**1**	**440**	**15**	**7**	**0.22**
`rectangulo2.dzn`	1. Base (solo `no_solapar`)	3,840*	43,645,037	657,009	324,660	>5,000
$(8 \times 5, n = 9)$	2. Base + Redundantes (`cumulative`)	4,778*	564,913	15,461	2,948	>5,000
	3. Base + Simetrías	8	78,857	1,505	745	4.39
	4. Completo (Redundantes + Simetrías)	**8**	**50,329**	**753**	**369**	**4.16**
`rectangulo3.dzn`	1. Base (solo `no_solapar`)	1,986*	18,725,217	280,758	138,388	>5,000
$(10 \times 7, n = 12)$	2. Base + Redundantes (`cumulative`)	2,375*	221,080	4,910	75	>5,000
	3. Base + Simetrías	122	1,254,766	21,755	10,756	114.01
	4. Completo (Redundantes + Simetrías)	**122**	**646,885**	**9,649**	**4,703**	**103.03**

Tabla 7: Estudio de ablación en búsqueda exhaustiva (`--all-solutions`, $*$ indica corte a los 5 s).

Asimismo, el Cuadro 8 compara las seis estrategias de búsqueda para encontrar la primera solución factible, contrastando el Modelo Base (Configuración 1) frente al Modelo Completo (Configuración 4).

		Modelo Base (Config. 1)			Modelo Completo (Config. 4)		
Instancia	Estrategia de Búsqueda	Tiempo (ms)	Nodos	Fallos	Tiempo (ms)	Nodos	Fallos
`rectangulo1.dzn`	Default (`solve satisfy`)	0.14	14	4	0.18	8	3
	`input_order + indomain_min`	0.13	13	3	**0.13**	**4**	**0**
	`input_order + indomain_max`	0.14	13	3	0.16	15	7
	`first_fail + indomain_min`	0.15	18	6	0.15	6	1
	`seq_search (x luego y, ff)`	0.19	35	14	0.19	**4**	**0**
	`dom_w_deg + indomain_split`	0.15	12	2	0.18	5	1
`rectangulo2.dzn`	Default (`solve satisfy`)	1.18	322	156	**0.28**	**18**	6
	`input_order + indomain_min`	0.29	69	29	0.28	20	5
	`input_order + indomain_max`	0.25	69	29	1.09	247	122
	`first_fail + indomain_min`	191.72	76,986	38,488	0.31	32	11
	`seq_search (x luego y, ff)`	521.30	238,330	119,160	0.30	29	10
	`dom_w_deg + indomain_split`	0.63	186	85	0.33	**18**	**4**
`rectangulo3.dzn`	Default (`solve satisfy`)	0.67	147	66	0.48	43	17
	`input_order + indomain_min`	0.63	135	60	**0.29**	**22**	**4**
	`input_order + indomain_max`	0.72	135	60	2.86	407	199
	`first_fail + indomain_min`	813.19	257,007	128,496	0.63	43	16
	`seq_search (x luego y, ff)`	2,795.07	807,419	403,704	0.58	51	19
	`dom_w_deg + indomain_split`	0.65	94	34	0.35	30	8

Tabla 8: Evaluación experimental de estrategias de búsqueda en colocación de cuadrados (primera solución).

6.5. Conclusiones del Ejercicio

- **Efecto crítico de la restricción global `cumulative`:** En el Modelo Base (que depende exclusivamente de las disyunciones binarias `no_solapar`), heurísticas como `first_fail` y `seq_search` sufren un estancamiento severo en `rectangulo3.dzn` (257,007 y 807,419 nodos respectivamente), ya que al fijar coordenadas en un solo eje sin verificar la capacidad ortogonal acumulada construyen configuraciones globalmente infactibles. Al activar `cumulative` en X e Y, los nodos con `first_fail` caen de 257,007 a 39 (una reducción del

99,98 %).

- **Superioridad de la heurística *Largest-Square-First* (`input_order + indomain_min`):** Al ordenar los cuadrados de mayor a menor tamaño e intercalar $[x_i, y_i]$, la estrategia `input_order + indomain_min` junto con el modelo completo logra el mejor desempeño global (4 nodos y 0 fallos en `rectangulo1.dzn`, y solo 22 nodos y 4 fallos en `rectangulo3.dzn`). Por el contrario, `indomain_max` degrada el rendimiento en el modelo con rompimiento de simetrías porque intenta asignar valores altos de (x, y) que entran en conflicto directo con el orden lexicográfico ascendente $x_i \leq x_{i+1}$ y la restricción del cuadrante inferior izquierdo.

7. Conclusiones Generales

- **Modelado CSP y Expresividad:** ...
- **Poda mediante Restricciones Redundantes:** ...
- **Rompimiento de Simetrías:** ...
- **Estrategias de Búsqueda y Distribución:** ...